

Laboratório Prático: ETL Financeiro com Apache Airflow

Aula 17 - Extração e Preparação de Dados

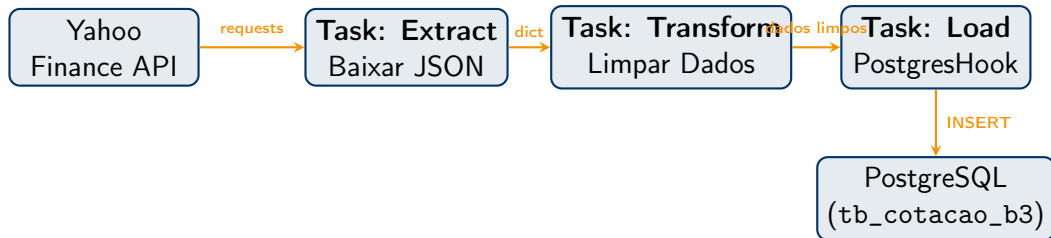
Ciências de Dados e Inteligência Artificial

IBMEC

14 de maio de 2026

- **Cenário Prático:** Extrair a cotação atual de uma ação da B3 (ex: PETR4) via API e armazenar em um Banco de Dados Relacional.
- **Desafios Abordados:**
 - Realizar Web Scraping / API Calls com bloqueio de User-Agent.
 - Navegar em payloads JSON complexos e aninhados.
 - Orquestrar tarefas de Extração, Transformação e Carga (ETL).
 - Salvar em um banco PostgreSQL usando PostgresHook.
- **Orquestrador:** Apache Airflow 3.x (TaskFlow API e PythonOperator).

Arquitetura do Pipeline



Conceito Chave

O Airflow orquestra o fluxo de dados em um Grafo Acíclico Dirigido (DAG), garantindo que a transformação só aconteça se a extração tiver sucesso.

O Desafio da Extração (Extract)

Ao acessar o Yahoo Finance via código, somos frequentemente bloqueados (Erro 403) porque o servidor rejeita scripts não identificados.

Solução: Forjar um User-Agent de navegador.

```
1 @task
2 def extrair_preco_acao() -> dict:
3     ticker = "PETR4.SA"
4     url = f"https://query2.finance.yahoo.com/v8/finance/chart/{ticker}"
5
6     headers = {
7         'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
8     }
9
10    resposta = requests.get(url, headers=headers)
11    resposta.raise_for_status()
12    return resposta.json()
```

O Desafio da Transformação (Transform)

Os dados retornam em um JSON com várias camadas: chart -> result -> [0] -> meta -> regularMarketPrice.

```
1 @task
2 def transformar_dados(dados_brutos: dict) -> dict:
3     meta_dados = dados_brutos['chart']['result'][0]['meta']
4     preco_atual = float(meta_dados['regularMarketPrice'])
5     simbolo = meta_dados['symbol']
6     moeda = meta_dados['currency']
7
8     return {
9         'ativo': simbolo.replace('.SA', ''), # 'PETR4.SA' vira 'PETR4'
10        'preco_brl': preco_atual,
11        'moeda': moeda,
12        'data_hora': datetime.now().strftime("%Y-%m-%d %H:%M:%S")
13    }
```

A Carga no Banco (Load) com PostgresHook

Em vez de gerenciar senhas e conexões manualmente (como usar `psycopg2` direto), utilizamos a funcionalidade nativa do Airflow: as **Connections** (via Admin > Connections).

```
1 @task
2 def carregar_banco(dados_finais: dict):
3     # O Airflow gerencia as credenciais sob o ID 'postgres_default'
4     hook = PostgresHook(postgres_conn_id='postgres_default')
5
6     sql_insert = """
7         INSERT INTO tb_cotacao_b3 (ticker, preco_brl, moeda, horario_cotacao)
8         VALUES (%s, %s, %s, %s);
9     """
10
11     # O Hook injeta os parametros com segurança para evitar SQL Injection
12     hook.run(sql_insert, parameters=(
13         dados_finais['ativo'], dados_finais['preco_brl'],
14         dados_finais['moeda'], dados_finais['data_hora']
15     ))
```

Comparativo: Clássico vs TaskFlow API

A evolução da orquestração no Airflow: como compartilhar dados entre as tarefas?

Abordagem Clássica (PythonOperator):

```
1 # Exige o uso manual de ti (Task Instance) e XComs
2 def transformar(**kwargs):
3     dados_brutos = kwargs['ti'].xcom_pull(task_ids='extrair', key='raw')
4     # ... processa ...
5     kwargs['ti'].xcom_push(key='clean', value=dados_limpos)
```

TaskFlow API (Airflow 2.0+):

```
1 # O Airflow gerencia os XComs nos bastidores automaticamente
2 dados_brutos = extrair_preco_acao()
3 dados_limpos = transformar_dados(dados_brutos)
4 carregar_banco(dados_limpos)
```

Agendamento Contínuo (CRON)

Para transformar este script em um processo diário, basta ajustar o agendamento da DAG.

- `schedule='@hourly'`: Roda na virada de cada hora.
- `schedule='0 10 * * *'`: Roda todo dia às 10h00 da manhã.
- `schedule='0 10-17 * * 1-5'`: Roda de hora em hora entre 10h e 17h, de segunda (1) a sexta (5).

```
1 @dag(  
2     dag_id='pipeline_etl_financas_b3',  
3     start_date=datetime(2026, 5, 10),  
4     schedule='0 10-17 * * 1-5', # Formato CRON  
5     catchup=False  
6 )  
7 def financas_etl_pipeline():  
8     # ...
```

O que acontece se a API estiver fora do ar ou retornar None?

- 1 **Sensores (Sensors):** Usaremos o `HttpSensor` para consultar a disponibilidade da rede antes de extrair os dados.
- 2 **Validação de Regra de Negócio:** Criaremos etapas para confirmar que $Preço > 0$, caso contrário, a DAG precisa disparar um alerta e abortar a carga no banco.

Vamos colocar a mão na massa!